

Robotic Automation System

Unit-1: Introduction to Embedded System

An **Embedded System (ES)** is a specialized computing system designed to perform **dedicated functions**, often within a larger system. It is a combination of **hardware + software + sometimes mechanical parts**, optimized for specific real-time applications.

Characteristics

- **Single-purpose:** Designed for specific tasks.
- **Real-time operation:** Deterministic response.
- **Low power consumption.**
- **Compact size.**
- **High reliability.**
- **Application-specific hardware and software.**
- **Often resource-constrained** (memory, processing, energy).

Examples

- Washing machines
- Smartphones
- Medical devices
- Automotive engine control
- IoT gadgets

2. Design of Embedded Systems

Embedded system design involves **co-design of hardware, software, and interfaces** to meet functional and non-functional requirements.

Design Steps

1. **Requirement Analysis**
 - Functional requirements
 - Timing constraints
 - Environmental constraints
 - Power, cost, reliability
2. **System Specification**
 - High-level architecture
 - Interfaces
 - Performance targets
3. **Hardware–Software Partitioning**
 - Decide which tasks run on hardware and which on software.
4. **Hardware Design**
 - Processor selection (8-bit/16-bit/32-bit MCU/MPU)
 - Memory design (ROM/RAM)
 - I/O interface design
 - Sensors, actuators

5. **Software Design**
 - RTOS or bare-metal programming
 - Drivers
 - Communication protocols
 6. **Integration and Testing**
 - Functional testing
 - Real-time testing
 - Stress and reliability tests
 7. **Deployment and Maintenance**
-

3. Categories of Embedded Systems

Embedded systems can be classified based on performance, complexity, and application domain.

A. Based on Performance

1. **Small-Scale Embedded Systems**
 - 8-bit/16-bit microcontrollers
 - Uses assembly/C
 - Examples: Remote controls, toys
2. **Medium-Scale Embedded Systems**
 - 32-bit microcontrollers
 - Uses RTOS
 - Examples: Automotive systems, mobile phones
3. **Large-Scale Embedded Systems**
 - High-speed processors, DSPs, FPGAs
 - Complex OS (Linux, Android)
 - Examples: Smart appliances, routers, robots

B. Based on Real-Time Requirements

1. **Hard Real-Time Systems**
 - Strict deadlines
 - Example: Anti-lock braking systems, pacemakers
2. **Soft Real-Time Systems**
 - Missed deadlines tolerated
 - Example: Video streaming, gaming

C. Based on Application Domain

- Automotive
- Consumer electronics
- Aerospace
- Medical electronics
- Industrial automation
- Telecom systems

4. Overview of Embedded System Architecture

A typical embedded system architecture has four layers:

A. Hardware Layer

- Processor (MCU/MPU/DSP/FPGA)
- Memory (RAM, ROM, Flash)
- I/O ports
- Timers/Counters
- Peripherals (ADC, DAC, UART, SPI, I2C)

B. System Software Layer

- Firmware
- Device drivers
- RTOS kernel (scheduler, memory mgmt, IPC)

C. Middleware Layer

- Communication stacks (TCP/IP, Bluetooth, Zigbee)
- File systems
- Libraries

D. Application Layer

- User interface
- Application logic
- Control algorithms

5. Recent Trends in Embedded Systems

1. Internet of Things (IoT)

- Connected embedded devices
- Cloud-based control and analytics

2. Artificial Intelligence at the Edge

- Edge AI processors
- TinyML (Machine learning on microcontrollers)

3. Embedded Linux

- Popular for high-performance embedded devices
- Examples: Raspberry Pi, BeagleBone

4. Autonomous Systems

- Self-driving vehicles
- Drones

- Robotics

5. Low-Power Designs

- Ultra-low-power MCUs
- Energy harvesting systems

6. Cybersecurity in Embedded Systems

- Secure boot
- Trusted execution environments
- Hardware encryption

6. Hardware Architecture of Embedded System

Key Components

1. **Processor (CPU/MCU/MPU)**
 - Heart of the system
 - Handles control and computation
2. **Memory**
 - ROM (stores firmware)
 - RAM (temporary data)
 - Flash (non-volatile storage)
3. **I/O Interfaces**
 - Serial (UART, SPI, I2C)
 - Parallel I/O
 - GPIO pins
4. **Timers and Counters**
 - Real-time clock
 - Event timing
5. **Sensors and Actuators**
 - Sensors gather data
 - Actuators take action
6. **Power Supply Unit**
 - Regulated supply
 - Battery or mains
7. **Communication Modules**
 - WiFi, Bluetooth, GSM, NFC
8. **System Bus**
 - Connects CPU with peripherals

7. Real-Time Embedded Systems

Systems designed to respond **predictably and within a bounded time**.

Types

1. **Hard Real-Time**

- Absolutely must meet deadlines
- Example: Airbag control, nuclear plant control
- 2. **Firm Real-Time**
 - Occasional deadline misses tolerated
 - Example: Industrial robots
- 3. **Soft Real-Time**
 - Performance degrades slightly with delays
 - Example: Multimedia systems

RTOS Components

- Task scheduler
 - Inter-process communication (IPC)
 - Task management
 - Interrupt handling
-

8. Robots and Robotics

Robots

A robot is a reprogrammable machine capable of performing tasks autonomously or semi-autonomously.

Components

- Embedded controller (microcontroller or microprocessor)
- Sensors (vision, temperature, proximity)
- Actuators (motors, hydraulic systems)
- Communication
- Power system

Applications

- Industrial automation
- Healthcare (surgery robots)
- Military
- Domestic robots (vacuum cleaners)
- Agriculture

Robotics and Embedded Systems

Embedded systems control:

- Motor control
- Perception and sensing
- Manipulation
- Path planning
- Real-time decision making

9. Microprocessors and Microcontrollers

Microprocessor

- CPU only; external peripherals needed
- Used in high-performance computing
- Examples: Intel i5, ARM Cortex-A

Microcontroller

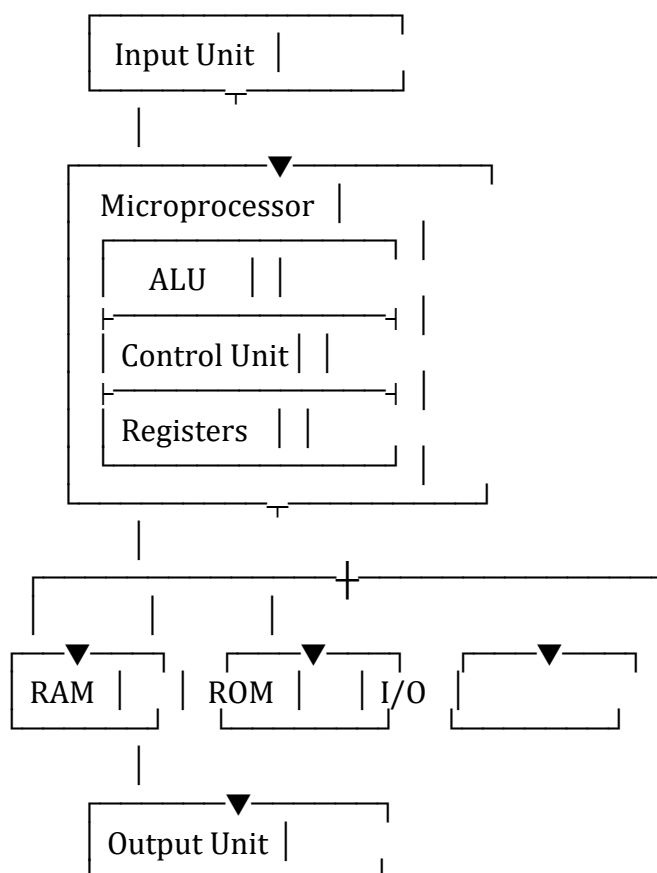
- CPU + memory + peripherals integrated on a single chip
- Designed for embedded applications
- Examples: 8051, AVR, PIC, ARM Cortex-M
- Cost-effective and low power

1. Microprocessor

Definition

A **microprocessor** is a programmable **CPU on a single chip** that performs arithmetic, logical, and control operations. It requires **external memory and I/O devices** to form a complete system.

Block Diagram of Microprocessor System



Characteristics

- CPU only on chip
- External RAM, ROM, and I/O required
- High speed processing
- Higher power consumption

Applications

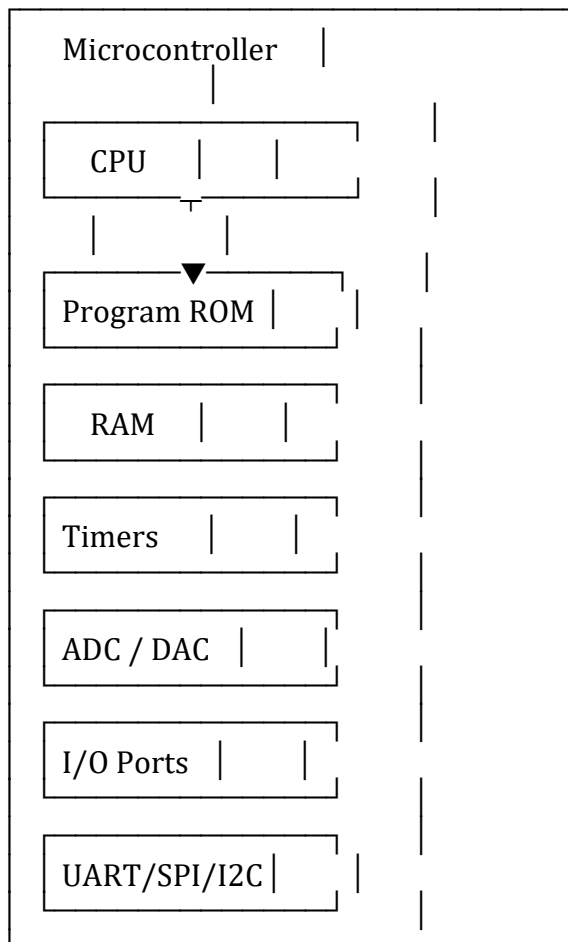
- Computers and laptops
- Servers
- High-performance systems

2. Microcontroller

Definition

A **microcontroller** is a **complete computer on a single chip** containing CPU, memory, I/O ports, timers, and communication interfaces. It is designed for **control-oriented applications**.

Block Diagram of Microcontroller



Characteristics

- CPU, memory, and peripherals on one chip
 - Low power consumption
 - Real-time control
 - Compact and cost-effective
-

Applications

- Embedded systems
 - Home appliances
 - Industrial automation
 - Robotics
-

3. Difference Between Microprocessor and Microcontroller

Feature	Microprocessor	Microcontroller
Integration	CPU only	CPU + Memory + I/O
External components	Required	Minimal
Power consumption	High	Low
Cost	High	Low
Real-time control	Poor	Excellent
Application	General purpose	Dedicated control

4. Microcontroller (Embedded Controller) Used in Robotics

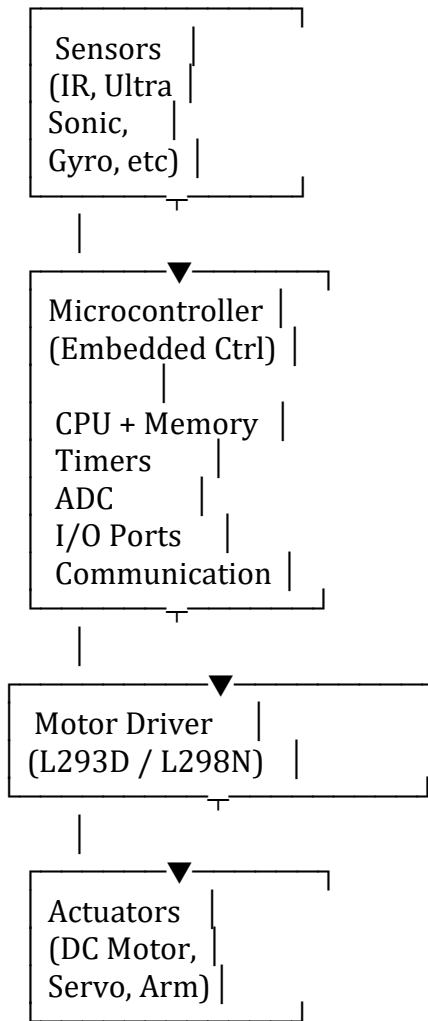
Definition

In robotics, a **microcontroller acts as an Embedded Controller** that **controls sensors, processes data, and drives actuators in real time.**

Advantages in Robotics

- ✓ Fast real-time control
- ✓ Low power consumption
- ✓ Compact size
- ✓ Easy hardware interfacing

Block Diagram of Robotic Embedded System



Functions of Microcontroller in Robotics

- Reads sensor data
- Processes control algorithms
- Controls motors and actuators
- Ensures real-time response
- Communicates with other systems

Common Microcontrollers Used in Robotics

- **Arduino (ATmega328)**
 - **PIC Microcontrollers**
 - **ARM Cortex-M (STM32)**
 - **RP2040 (Raspberry Pi Pico)**
-